Homework 8

DATA 624

Kevin Havis

November 8, 2025

Homework 8

7.2

For this assignment, we've been asked to experiment with different non-linear models, including SVM, MARS, and KNN.

The book provides an example for KNN which we will include as a benchmark, alongside which we will train the SVM and MARS models.

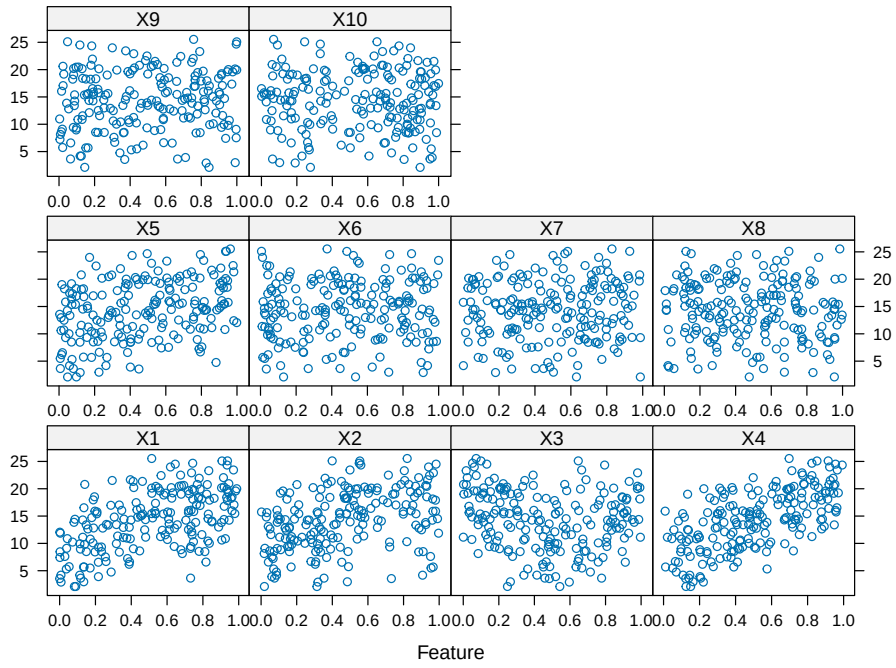
The convenient `mlbench` package allows us to generate data on the fly to experiment; some of the features are meaningful to the target variable, while others are added as noise. As such, we will check the feature importances in a later section.

Setting up the train / test splits, the cross validation, and the models are our first steps.

```
set.seed(42)


training_data <- mlbench.friedman1(200, sd=1)
training_data$x <- data.frame(training_data$x)

featurePlot(training_data$x, training_data$y)
```




```r
test_data <- mlbench.friedman1(5000, sd=1)
test_data$x <- data.frame(test_data$x)

# Set up cross validation
ctrl <- trainControl(method = 'cv', number = 5)

models <- caretList(
  x=training_data$x,
  y=training_data$y,
  trControl = ctrl,
  methodList = c("svmRadial", "earth", "knn"))

# Get predictions for all models
pred_svm <- predict(models$svmRadial, newdata = test_data$x)
pred_mars <- predict(models$earth, newdata = test_data$x)
pred_knn <- predict(models$knn, newdata = test_data$x)


# Compare performance
results <- data.frame(
  svm = postResample(pred_svm, test_data$y),
  earth = postResample(pred_mars, test_data$y),
```

```
  knn = postResample(pred_knn, test_data$y)
)

results |>
  kable()
```

	svm	earth	knn
RMSE	2.3212708	1.9044486	3.0817004
Rsquared	0.7887576	0.8575472	0.6482215
MAE	1.7818217	1.5111811	2.4434095

```
summary(models$earth)
```

```
Call: earth(x=data.frame[200,10], y=c(22.04,17.84,1...), keepxy=TRUE, degree=1,
            nprune=14)

                coefficients
(Intercept)        3.516867
h(0.514408-X1)   -11.830065
h(0.574637-X2)   -13.329399
h(X2-0.574637)    -4.110667
h(X3-0.174528)    21.846365
h(0.721888-X3)    24.406345
h(X3-0.721888)   -12.575955
h(X3-0.895891)    27.174405
h(X4-0.182561)    17.685517
h(X4-0.325131)    -8.225436
h(0.92534-X5)     -5.346359
h(0.422881-X10)   -1.921303

Selected 12 of 17 terms, and 6 of 10 predictors (nprune=14)
Termination condition: Reached nk 21
Importance: X4, X1, X2, X5, X3, X10, X6-unused, X7-unused, X8-unused, ...
Number of terms at each degree of interaction: 1 11 (additive model)
GCV 2.86722    RSS 449.1356    GRSq 0.9025661    RSq 0.9229184
```

For this dataset, MARS (a.k.a. `earth`) performs the best with an R2 of 0.8575, followed by SVM with 0.7887 and KNN of 0.6482. Generally, models with R2 greater than 80% are acceptable, so the only real choice would be the MARS model.

Check the features that MARS selected, we can see that the "real" features were indeed identified (X1-X5). Interestingly, it also selected X10 with a moderate coefficient, which isn't intended to be one of the useful features. This is likely due to random correlation in the dataset.

7.5

We will now apply the same principles to the dataset we previously looked at in Section 6.3, where we originally trained linear models against the data. This time, we'll see if non-linear can perform better.

I've include our best performing linear model, Lasso, for comparison. Last time we got an R2 of 0.6946, so that is our mark to beat with the non-linear models.

As we did previously, we will impute the missing data in this dataset using KNN for all models.

```
set.seed(42)


data(ChemicalManufacturingProcess)

# Split up the data set
y <- ChemicalManufacturingProcess[, 1]

x <- ChemicalManufacturingProcess[, -1]

# Use this partition to ensure well-distributed splits
idx <- createDataPartition(y, p = 0.8, list = FALSE)

train_x <- x[idx, ]
train_y <- y[idx]

test_x <- x[-idx, ]
test_y <- y[-idx]


# Use recipes to help us set up the preprocessing

rec <- recipe(y ~ ., data=data.frame(train_x, y=train_y)) |>
  step_impute_knn(all_predictors(), neighbors=5) #KNN null imputation
```

```r
# Set up cross validation
chem_ctrl <- trainControl(method = 'cv', number = 5)

models <- caretList(
  x=train_x,
  y=train_y,
  preProcess = 'knnImpute',
  trControl = chem_ctrl,
  methodList = c("svmRadial", "earth", "knn", "lasso"))

# Get predictions for all models
pred_svm <- predict(models$svmRadial, newdata = test_x)
pred_mars <- predict(models$earth, newdata = test_x)
pred_knn <- predict(models$knn, newdata = test_x)
pred_lasso <- predict(models$lasso, newdata = test_x)


# Compare performance
results <- data.frame(
  svm = postResample(pred_svm, test_y),
  earth = postResample(pred_mars, test_y),
  knn = postResample(pred_knn, test_y),
  lasso = postResample(pred_lasso, test_y)
)

results |>
  kable()
```

	svm	earth	knn	lasso
RMSE	1.1192555	1.2960443	1.3103492	1.1369818
Rsquared	0.7389966	0.5983647	0.5968886	0.6946560
MAE	0.8391098	1.0567249	0.9254375	0.9054753

As we can see, both SVM and MARS models quickly outperform the Lasso model for this data, with SVM having slightly higher R2 values. This tells me that the relationship for this data is likely not quite linear, given the improvement.

Inspecting the top ten features, the Manufacturing processes again provide the most value. The main difference however, is that none of the top 10 manufacturing process features besides the 10th appeared in our linear (lasso) model. This is intuitive as we would expect non-linear features to work better with non-linear models and vice-versa.

```r
svm_importance <- filterVarImp(x = train_x, y = train_y) %>%
  as_tibble(rownames = "feature") %>%
  arrange(desc(Overall))

svm_importance |> kable(caption="Top Non-Linear (SVM) Features")
```

Table 3: Top Non-Linear (SVM) Features

feature	Overall
ManufacturingProcess32	8.8150853
ManufacturingProcess09	7.9624396
ManufacturingProcess13	7.1939071
ManufacturingProcess36	6.9999203
ManufacturingProcess17	6.2958192
BiologicalMaterial03	5.5935262
BiologicalMaterial06	5.5006816
BiologicalMaterial02	5.3545095
ManufacturingProcess06	5.2024079
ManufacturingProcess33	4.6722402
ManufacturingProcess11	4.3420097
ManufacturingProcess12	4.3050363
BiologicalMaterial12	3.9072943
BiologicalMaterial08	3.8589620
BiologicalMaterial04	3.8117864
BiologicalMaterial01	3.7650980
BiologicalMaterial11	3.5416457
ManufacturingProcess04	2.6941393
ManufacturingProcess34	2.6846590
ManufacturingProcess30	2.5676792
ManufacturingProcess24	2.4101906
ManufacturingProcess28	2.3038972
ManufacturingProcess38	2.2007377
ManufacturingProcess10	2.1632566
ManufacturingProcess15	1.8857886
ManufacturingProcess35	1.7272780
ManufacturingProcess37	1.6747738
ManufacturingProcess43	1.6597315
BiologicalMaterial10	1.6407830
BiologicalMaterial09	1.5376009
BiologicalMaterial07	1.5063039
ManufacturingProcess02	1.3907708

feature	Overall
ManufacturingProcess05	1.3617723
ManufacturingProcess29	1.3043781
ManufacturingProcess01	1.2094065
ManufacturingProcess23	1.0745056
ManufacturingProcess20	0.9041027
ManufacturingProcess42	0.8868761
ManufacturingProcess21	0.8418763
ManufacturingProcess19	0.8123931
ManufacturingProcess18	0.8070471
ManufacturingProcess03	0.6486808
BiologicalMaterial05	0.6327042
ManufacturingProcess31	0.5920345
ManufacturingProcess16	0.5673586
ManufacturingProcess14	0.5593035
ManufacturingProcess07	0.4394821
ManufacturingProcess44	0.4198801
ManufacturingProcess26	0.3636097
ManufacturingProcess08	0.2402531
ManufacturingProcess45	0.2359176
ManufacturingProcess22	0.1766514
ManufacturingProcess40	0.1038609
ManufacturingProcess25	0.0575012
ManufacturingProcess39	0.0406126
ManufacturingProcess41	0.0280836
ManufacturingProcess27	0.0276532

```
# Extract selected coefficients
lasso_coefs <- models$lasso$finalModel$beta.pure %>%
  as_tibble(rownames = "step") %>%
  slice(n()) %>%
  pivot_longer(-step, names_to = "predictor", values_to = "coefficient") %>%
  filter(coefficient != 0) %>%
  select(predictor, coefficient) %>%
  arrange(desc(abs(coefficient)))

kable(lasso_coefs, caption="Top Linear (Lasso) Features")
```

Table 4: Top Linear (Lasso) Features

predictor	coefficient
ManufacturingProcess25	4.2313610
ManufacturingProcess26	-4.1426877
ManufacturingProcess18	2.0982384
ManufacturingProcess20	-2.0337182
ManufacturingProcess32	1.7480257
ManufacturingProcess29	1.6618728
BiologicalMaterial03	1.4736848
ManufacturingProcess27	-1.3686927
ManufacturingProcess33	-0.9242963
BiologicalMaterial04	-0.8888528
BiologicalMaterial09	-0.7297452
ManufacturingProcess11	0.6990133
BiologicalMaterial02	-0.6237019
BiologicalMaterial10	0.6192414
ManufacturingProcess14	-0.5924019
BiologicalMaterial06	-0.5826265
ManufacturingProcess28	-0.5471846
ManufacturingProcess09	0.5322210
BiologicalMaterial12	0.5105365
ManufacturingProcess15	0.5053203
ManufacturingProcess45	0.4500639
BiologicalMaterial11	-0.3859492
BiologicalMaterial08	0.3433081
ManufacturingProcess10	-0.3356440
ManufacturingProcess38	-0.3343907
ManufacturingProcess42	0.3322267
ManufacturingProcess30	-0.3288150
ManufacturingProcess04	0.3151780
ManufacturingProcess19	0.3093032
ManufacturingProcess37	-0.2934197
BiologicalMaterial01	0.2410137
ManufacturingProcess01	0.2285123
ManufacturingProcess12	0.2033769
ManufacturingProcess13	-0.1973505
ManufacturingProcess36	0.1882575
ManufacturingProcess24	-0.1755778
BiologicalMaterial07	-0.1633496
ManufacturingProcess44	-0.1602868
ManufacturingProcess35	-0.1547985

predictor	coefficient
ManufacturingProcess22	-0.1482617
ManufacturingProcess43	0.1413384
ManufacturingProcess39	0.1344628
ManufacturingProcess07	-0.1242289
BiologicalMaterial05	0.1194050
ManufacturingProcess34	-0.0980845
ManufacturingProcess06	0.0535838
ManufacturingProcess02	0.0449110
ManufacturingProcess08	-0.0422451
ManufacturingProcess41	0.0342556
ManufacturingProcess21	0.0248343
ManufacturingProcess05	0.0228573
ManufacturingProcess40	0.0181970
ManufacturingProcess16	-0.0161102
ManufacturingProcess03	0.0108719
ManufacturingProcess23	0.0102578
ManufacturingProcess31	0.0053574

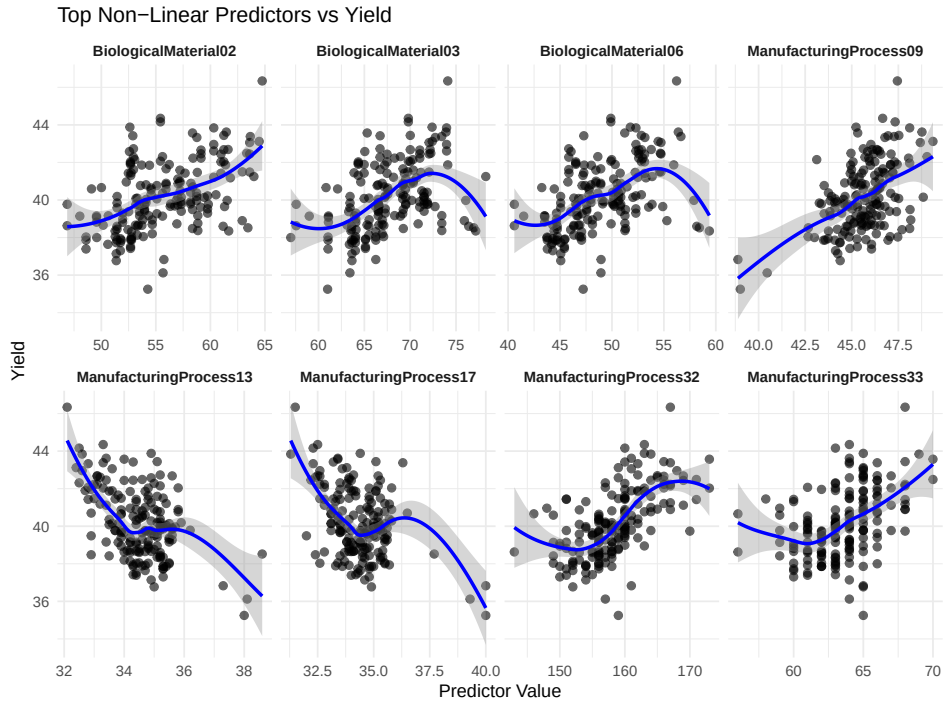
Finally, visually inspecting the relationship between the strong predictors and the target variable confirm our intuition regarding the appropriateness of a linear vs non-linear model. I've also included the common predictor, `ManufacturingProcess33`, since it appeared mutually; given that the majority of the line of best fit is approximately linear, and appears to be relatively well correlated, it is reasonable for both models to find it informative.

```
top_preds <- c("ManufacturingProcess32", "ManufacturingProcess09",
               "ManufacturingProcess13", "ManufacturingProcess36",
               "ManufacturingProcess17", "BiologicalMaterial03",
               "BiologicalMaterial06", "BiologicalMaterial02", "ManufacturingProcess33")

plot_data <- x[, top_preds] %>%
  as_tibble() %>%
  bind_cols(yield = y) %>%
  pivot_longer(-yield, names_to = "predictor", values_to = "value") %>%
  drop_na() %>%
  filter(value >= 1)

ggplot(plot_data, aes(x = value, y = yield)) +
  geom_point(alpha = 0.6, size = 2) +
  geom_smooth(method = "loess", se = TRUE, color = "blue") +
  facet_wrap(~predictor, scales = "free_x", ncol = 4) +
```

```
  labs(title = "Top Non-Linear Predictors vs Yield",
       x = "Predictor Value",
       y = "Yield") +
  theme_minimal() +
  theme(strip.text = element_text(face = "bold"))
```



Conclusion

Knowing when to apply different models given different data is a key aspect of machine learning. There is no one-size fits all approach; we are always managing the bias variance trade off. While transforming the data to apply a simpler linear regression can be an impactful technique, sometimes this approach can feel like "square hole, round peg". The domain, as always, is the most important consideration, and a non-linear model may be the best way to model the data. As data scientists, this is where our experience and knowledge comes into play.